Programski prevodioci

03 Sintaksna analiza

25-26

Dunja Vrbaški

Fakultet tehničkih nauka, Novi Sad

2 osnovna tipa parsera:

Silazno parsiranje (top-down)

Pokušavamo da od početnog pojma izvedemo ulazni niz simbola prateći pravila produkcije
Akcije: Leva strana pravila se zamenjuje desnom

Uzlazno parsiranje (bottom-up)

Pokušavamo da od ulaznog niza simbola redukcijom dobijemo početni pojam
Akcije: Desna strana pravila se zamenjuje levom

TD – jednostavnije za direktnu implementaciju
BU – širu klasu gramatika podržavaju

Silazno parsiranje (top-down)

Pokušavamo da, od početnog pojma, izvedemo ulazni niz simbola prateći pravila produkcije
Akcije: Leva strana pravila se zamenjuje desnom

text
→ *sentence*
→ *text sentence*

Recursive descent parser

Rekurzivni spust

- Postoji funkcija za svaki pojam odgovorna za obradu tog pojma
- Početna funkcija odgovara početnom pojmu i ona poziva odgovarajuće funkcije (za pojmove sa desne strane pravila)
- Rekurzivne funkcije zbog rekurzivne prirode gramatike PL
- Algoritam je intuitivan i relativno jednostavan za direktnu implementaciju (ručno)

Problemi

text
→ *sentence*
→ *text sentence*

Šta raditi kod alternativa – koju pozvati? Odrediti heuristiku.

Šta je greška? – kraj izvršavanja pre kraja ulaznog teksta, pogrešan izbor alternative?

Šta raditi?
- Backtracking ili
- Nešto složenije, ali pametnije (predictive parsing, First and Follow skupovi, LL(k),...)

- Leva rekurzija može biti problem - postoje heurisitike za uklanjanje rekurzije.

Uzlazno parsiranje (bottom-up)

Pokušavamo da od ulaznog niza simbola, redukcijom, dobijemo početni pojam
Akcije: Desna strana pravila se zamenjuje levom

shift-reduce algoritam

- Preuzima se jedan simbol (token) iz ulaznog niza *(shift)*
- Dodaje se na kraj prethodno formirane sekvence pojmova/simbola
- Proverava se da li novoformirana sekvenca odgovara desnoj strani nekog pravila
    - ako odgovara – zamenjuje se čitava sekvenca pojmom sa leve strane *(reduce)*
    - ako ne odgovara – nastavlja se preuzimanje

→ Pravila izvođenje se otkrivaju u obrnutom redosledu od top-down.

„Ovo je tekst.“

CAPITAL_WORD WORD WORD DOT EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

„Ovo je tekst.“

WORD WORD DOT EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

CAPITAL_WORD

„Ovo je tekst.“

WORD WORD DOT EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

CAPITAL_WORD

„Ovo je tekst.“

WORD WORD DOT EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

words

„Ovo je tekst.“

WORD DOT EOF

WORD
words

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

„Ovo je tekst.“

WORD DOT EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

WORD
words

„Ovo je tekst.“

WORD DOT EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

words

„Ovo je tekst.“

DOT EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

WORD
words

„Ovo je tekst.“

DOT EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

WORD
words

„Ovo je tekst.“

DOT EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

words

„Ovo je tekst.“

EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

DOT
words

„Ovo je tekst.“

EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

DOT
words

„Ovo je tekst.“

EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

sentence

„Ovo je tekst.“

EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

sentence

„Ovo je tekst.“

EOF

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

text

„Ovo je tekst.“

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

EOF
text

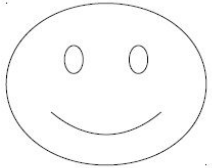
„Ovo je tekst.“

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD

EOF
text

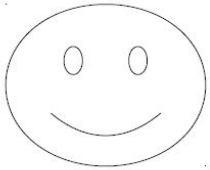


„Ovo je tekst.“

text
→ *sentence*
→ *text sentence*

sentence
→ *words* DOT

words
→ CAPITAL_WORD
→ *words* WORD
→ *words* CAPITAL_WORD



Obratiti pažnju:

- redukcija - desna strana se menja levom
- koristimo stek

bison

Bison je alat za generisanje parsera.

Na osnovu gramatike sintakse nekog jezika generiše funkciju koja predstavlja parser za taj jezik. Jednostavno se integriše sa flexom koji se koristi za leksičku analizu.

- bottom-up
- shift-reduce
- moguće gramatike su podskup kontekstno slobodnih gramatika

Regularnim jezicima (gramatikama) odgovaraju konačni automati (finite automata)
CF jezicima (gramatikama) odgovaraju potisni automati (pushdown automata)

Flex – koristi za automatsko generisanje skenera
Bison - koristi za automatsko generisanje parsera

Simuliraju rad automata koji prepoznaju stringove jezika (leksika, sintaksa) generisanog odgovarajućom gramatikom.

Ideja

MI: Sintaksa se predstavi gramatikom, formalno

TEORIJA: Za CFG postoji odgovarajući potisni automat (konačni automat + stek/memorija)

ALAT: Formira odgovarajući automat koji će biti zadužen za prepoznavanje

→ imamo mašinu/alat/implementaciju za prepoznavanje programskih konstrukcija i prihvatanje ulaznog programskog koda

Postupak

1. Definisati tokene
2. Definisati gramatiku sintakse
3. Povezati flex i bison
4. Omogućiti da flex vraća odgovarajuće tokene
5. Kompajlirati (u određenom redosledu) obe stvari da se dobiju skener i parser
6. Pokrenuti

Postupak

1. Definisati tokene (`jezik.y`)
2. Definisati gramatiku sintakse (`jezik.y`)
3. Povezati flex i bison (include + deklaracije)
4. Omogućiti da flex vraća odgovarajuće tokene (`jezik.l` → return IF)
5. Kompajlirati (u određenom redosledu) obe stvari da se dobiju skener, parser i main (`make`)
   `lex.yy.c` sa `yylex()`
   `jezik.tab.c` sa `yyparse()`
6. Pokrenuti

jezik.l

```
#include "syntax.tab.h"

%%
... regularni izrazi ...

"if" { return _IF; }

%%

int main() {
    yylex();
}
```

jezik.y

```
int yylex(void);
...

%token _IF
%token _ID
...

%%
... gramatika ...

if_statement...

%%

int main() {
    return yyparse();
}
```

```
...

%%
... gramatika ...

if_statement...   { printf("..."); }

%%

int main() {
   return yyparse();
}
```

Možemo dodati korisničke akcije

```
text
   → sentence
   → text sentence

sentence
   → words dot

words
   → capital_word
   → words word
   → words capital_word

word
   → small_letter
   → word small_letter

capital_word
   → capital_letter
   → capital_word small_letter

dot
   → "."
```

```
capital_letter
   →  "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H"
    | "I" | "J" | "K" | "L" | "M" | "N" | "O" | "P"
    | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X"
    | "Y" | "Z"
small_letter
   →  "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h"
    | "i" | "j" | "k" | "l" | "m" | "n" | "o" | "p"
    | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x"
    | "y" | "z"
```

```
"."          { return DOT; }

[A-Z][a-z]*  { return CAPITAL_WORD; }

[a-z]+       { return WORD; }
```

jezik.l

```
text
  : sentence
  | text sentence
  ;

sentence
  : words DOT
  ;

words
  : CAPITAL_WORD
  | words CAPITAL_WORD
  | words WORD
  ;
```

jezik.y

Možemo zadati i sledeći zadatak:
Prebrojati koliko ima reči i rečenica u ulaznom tekstu.

```
text
  : sentence
  | text sentence
  ;

sentence
  : words DOT
      { sentence_counter++; }
  ;
```

```
words
  : CAPITAL_WORD
      { word_counter++; }
  | words CAPITAL_WORD
      { word_counter++; }
  | words WORD
      { word_counter++; }
  ;
```

Dodajemo korisničke akcije. Gde ispisujemo ove vrednosti?